

SimQC: A Browser-Based Quantum Circuit Simulator with Stepwise Inspection and AI-Assisted Explanation

Aoutul Nabi Purna*, Shishir Dhar[†], Shihab Sharar[‡], Md. Iftikhar Alam Hrithik[§]

Department of Electrical and Computer Engineering

North South University, Dhaka, Bangladesh

CSE 299, Section 3, Group No. 1

Emails: {aoutul.purna.241, shishir.dhar.241, shihab.sharar.241, iftikhar.hrithik.241}@northsouth.edu

Abstract—This proposal presents a browser-based quantum circuit simulator built entirely in JavaScript without relying on existing quantum software development kits such as Qiskit [3], Cirq [4], or PennyLane [5]. The system represents multi-qubit states as complex amplitude vectors and applies gates only to their target qubits rather than constructing the full $2^n \times 2^n$ unitary matrix, which keeps the per-gate cost at $O(2^n)$ instead of $O(4^n)$ [11], [12]. The frontend provides a custom drag-and-drop circuit editor, while a separate simulation core handles state evolution, measurement, and canonical algorithms. Planned novel features include stepwise inspection with breakpoints, reusable subcircuit blocks, distinct entanglement highlighting, and an AI module that can either explain an existing circuit or synthesize one from a textual description. The committed algorithmic target is at least three of Deutsch–Jozsa [14], Grover [15], Shor [16], and the Quantum Fourier Transform [17]. This project will help clarify the inner workings of a quantum circuit and its functions, which otherwise seems like a complete black box to newcomers in the field. The purpose of this project is also to learn the inner workings of quantum circuits and algorithms in depth as we implement it through its developmental stages.

Index Terms—quantum computing, circuit simulator, state-vector simulation, drag-and-drop editor, JavaScript, AI-assisted explanation

I. INTRODUCTION

Quantum computing introduces computational primitives that have no direct classical equivalent, and even small circuits can produce behavior that is hard to predict by inspection [6]. Furthermore, the abstract linear algebra concepts and related mathematics fails to properly build an intuition about the actual mechanism and the inner working behind the circuits [7]. For students and practitioners who do not have access to quantum hardware, a simulator is the primary tool for developing intuition and verifying algorithms [8], [9]. Existing simulators vary widely in scope, performance, and licensing; many are tightly coupled to a particular SDK and assume either a Python environment or a server-side backend [3]–[5]. A lightweight, browser-resident simulator that can be opened without installation lowers the barrier to experimentation and lets a learner focus on the circuit itself rather than on environment setup [1], [10].

This project proposes such a simulator. The implementation uses no quantum SDK and no server-side runtime: quantum

states are represented explicitly as vectors of complex numbers, gates are applied as matrix operations on those vectors, and the entire program runs in the browser [1], [11]. Our team is targeting a working simulator with a fully custom drag-and-drop circuit editor and at least three canonical algorithms verified against their theoretical results [14], [15], [17]. Two additional capabilities distinguish the proposal from a minimal teaching tool: stepwise inspection of intermediate states [29], and an AI module that can either explain a circuit or build one from a natural-language description [2], [32], [33].

The remainder of this report states the problem, situates the work relative to existing simulators, lists the objectives and proposed novel features, describes the architecture and methodology, and outlines the validation and development plan. Throughout, claims that depend on external evidence are either supported by a citation or marked with a placeholder so that the team can fill them in before submission.

II. PROBLEM STATEMENT

The evolution that a small quantum system goes through under a short sequence of gate operations can be reasoned about, and computed, by hand: each gate is a small unitary matrix, and the state is a vector whose dimension is a power of two [6]. As the number of qubits grows, however, the state dimension grows exponentially, and multi-qubit behaviour, particularly entanglement and the interference patterns produced by algorithms such as Grover and the Quantum Fourier Transform, becomes difficult to understand and verify by inspection [15], [17], [27]. A numerically correct simulator is therefore needed that (i) represents multi-qubit states explicitly, (ii) applies gates efficiently to the targeted qubits rather than to the entire system [11], [13], and (iii) implements and verifies canonical quantum algorithms through the application of quantum gates, while (iv) letting users inspect how a circuit produces its result rather than only its final measurement [1], [29]. The lack of easy to understand simple simulators also leaves behind a gap in the concepts of beginners in the field of QC. The AI features in our project, specifically the AI explanations of the circuit, will help newbies bridge their understanding to build mental

models of state transitions and debug instead of acting as a regular simulation utility [32], [34].

Furthermore, understanding a new paradigm of computing can be difficult, especially when the field is in its infancy and lacks structured resources for onboarding of new students and researchers in the field. The project demonstrates step-wise changes to the quantum state and as a result will help provide clarity. The problem to be solved is also to gain an in depth understanding of quantum circuits, logical operations, and quantum algorithms, helping us bridge the gaps in understanding so that we can pursue our long term goals of research in quantum computing, digital logic and computer organization and architecture [6].

III. RELATED WORK

The principal reference for this project is *Quirk* [1], an open-source browser-based quantum circuit simulator. Quirk provides a drag-and-drop circuit grid with real-time simulation, inline state displays (probability, Bloch sphere, amplitude, density matrix), circuit serialization through the URL, undo/redo, and support for controlled gates [25], [26]. The present project uses Quirk as a design inspiration and does not copy its architecture. It is worth noting that Quirk already displays state after every circuit column [1], which partially overlaps with the step-inspection feature proposed in Section V; we therefore intend to scope to breakpoint-based, user-driven stepwise debugging rather than to per-column state display in general [29], [30].

IV. OBJECTIVES AND SCOPE

The project’s committed objectives are as follows. First, to implement an explicit representation of n -qubit state vectors as complex amplitudes, with normalization enforced after every gate, with a warning at 20 qubits and a total allowed maximum of 24 qubits [11], [12]. Second, to provide a gate library containing at minimum the Pauli gates (X , Y , Z), the Hadamard gate H , the controlled-NOT gate, and the rotation gates R_x , R_y , R_z [6], [18]. An extended set comprising I , S , T , CZ , $SWAP$, and Toffoli is planned but treated as an extension rather than a committed deliverable [6], [19]. Third, to apply each gate only to its target qubits so that the full $2^n \times 2^n$ unitary matrix is never materialized [11], [13]. Fourth, to implement measurement as repeated-shot sampling with state collapse [23], [24]. Fifth, to implement and verify at least three of the canonical algorithms the team has listed, Deutsch–Jozsa, Grover, Shor, and the Quantum Fourier Transform [14], [15], [17]. Sixth, to ship a fully custom drag-and-drop circuit editor that drives the simulation core through a clean interface. Finally to develop a (i) circuit explainer AI that will help describe and breakdown existing circuits, highlight patterns, simplify logic, etc. and (ii) a circuit builder AI that will help develop circuits from text input [32], [33].

The expected outcome is a working, browser-resident simulator that executes circuits correctly and demonstrates a minimum of three committed algorithms. The AI circuit builder

will be treated as a proposed feature, not as a guaranteed deliverable, and no claim is made about its accuracy or about any quantitative evaluation of it. Performance is discussed only in asymptotic terms (Section VII); no benchmark numbers are claimed.

V. PROPOSED NOVEL FEATURES

The features in this section are proposed as the project’s intended contributions beyond a minimal simulator.

Stepwise inspection with breakpoints. We aim to let a user place a breakpoint at a column of the circuit and execute step by step, observing the state after each gate [29], [30]. As noted in Section III, per-column state display already exists in Quirk [1]; our intended distinction is the breakpoint-style, debugger-oriented interaction model.

Reusable circuit blocks. A circuit may be saved and reused as a single subcircuit block in a larger circuit, with internal state hidden behind a defined interface [31]. When a simulation is paused, the user can select a subcircuit or add a breakpoint inside the subcircuit to inspect how that subset of gates transforms its current input.

Entanglement highlighting. Distinct visual cues aim to mark qubit pairs or groups whose joint state is entangled, helping users see structure that amplitude vectors alone do not surface [20], [27], [28].

Input/output analysis. Given the flattened (fully expanded) circuit, the user selects a set of input lines and a set of output lines; the AI module first analyses each input line independently, then explains how the per-input results combine to produce the selected outputs [32], [34].

Explainer AI and builder AI. The explainer takes a circuit and produces a textual description; the builder takes a textual description and produces a circuit [32], [33]. The builder AI is proposed features. No statement is made about either models’ accuracy and performance.

AI backend. The AI module is specified as one component with a swappable backend: either a remote API call or the team’s locally hosted model in Ollama [35]. The final backend choice is not yet fixed.

VI. SYSTEM ARCHITECTURE

The architecture follows a single principle: the user interface does not contain any mathematics and is essentially decoupled from the inner mathematical operations. Changes made to the circuit via interaction through UI will tell the simulation core something like “ H gate on qubit 2 at column 4”, and the core alone determines the mathematical meaning. As a consequence, the core must be fully functional with no UI attached, which is also a precondition for automated testing.

The system has two parts, shown in Fig. 1. The *frontend/UI* comprises the circuit editor, the gate toolbox, the state displays, the debugger, and the AI panel. The *simulation core* comprises the circuit data model, the quantum state engine, the gate library, the execution engine, the measurement module, and the algorithm implementations. A circuit is modelled as an ordered list of columns, each holding operations on specific

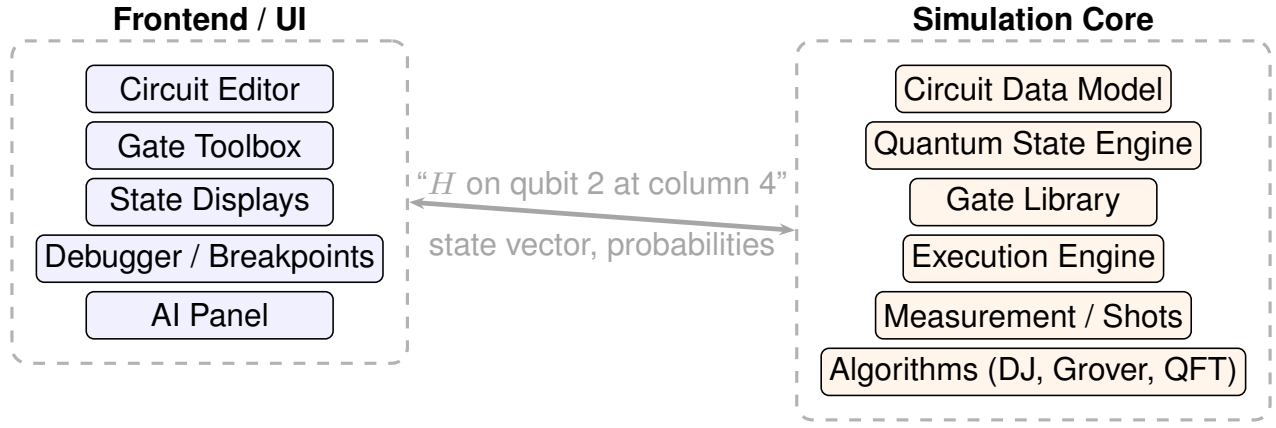


Fig. 1. Two-part architecture. The frontend contains no quantum mathematics; it issues high-level operations (e.g., “apply H on qubit 2 at column 4”) to the simulation core, which owns the state vector and the gate library. The core is usable with no UI attached.

qubits (gates, controls, or empty cells), and circuits serialize to JSON [31]. The dependency order inside the core is: complex numbers $\rightarrow$ quantum state $\rightarrow$ gate library $\rightarrow$ circuit executor $\rightarrow$ measurement, after which the editor and state displays are layered on, the full simulator is assembled, and algorithms are implemented on top [6], [11].

The planned source layout separates concerns into per-module directories: *core* (complex numbers and the quantum state), *gates*, *circuit*, *measurement*, *algorithms*, *visualization*, *editor*, *storage*, and *tests*. The first version of the displays targets probabilities, amplitudes, the state-vector in textual form, and a measurement histogram; the Bloch sphere and density matrix are deferred to a later iteration [25], [26].

VII. METHODOLOGY

An n -qubit pure state is represented as a normalized complex vector of dimension 2^n [6]:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad (1)$$

where each $\alpha_i \in \mathbb{C}$, $|i\rangle$ denotes the computational basis state indexed by the n -bit binary representation of i , and normalization requires

$$\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1. \quad (2)$$

Evolution under a gate (or a column of gates) is a unitary update [6], [36]:

$$|\psi_{k+1}\rangle = U_k |\psi_k\rangle. \quad (3)$$

Single-qubit operation in matrix form. For a single qubit, the state $|\psi\rangle = (\alpha, \beta)^\top \in \mathbb{C}^2$ and any gate G is a 2×2 unitary. The matrix–vector product that evolves the state is

$$G|\psi\rangle = \begin{pmatrix} g_{00} & g_{01} \\ g_{10} & g_{11} \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} g_{00}\alpha + g_{01}\beta \\ g_{10}\alpha + g_{11}\beta \end{pmatrix}. \quad (4)$$

For example, the Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ applied to $|0\rangle$ yields $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, the uniform superposition [6]. This is the canonical pattern for every gate application: a small matrix mixes a small set of amplitudes, and the cost is proportional to the size of that set.

Scaling to n qubits. The textbook construction lifts G to the full $2^n \times 2^n$ operator

$$U_t = \underbrace{I \otimes \cdots \otimes I}_t \otimes G \otimes \underbrace{I \otimes \cdots \otimes I}_{n-1-t}, \quad (5)$$

with G at position t via the Kronecker product [37], then computes $U_t |\psi\rangle$. This materializes a $2^n \times 2^n$ matrix and performs a dense matrix–vector product at cost $O(4^n)$ [11]. The present implementation avoids building U_t . Because U_t only mixes amplitude pairs whose indices differ in bit t (i.e., $j = i \oplus 2^t$), the update is performed pairwise on each such pair:

$$\begin{pmatrix} \alpha'_i \\ \alpha'_{i \oplus 2^t} \end{pmatrix} = G \begin{pmatrix} \alpha_i \\ \alpha_{i \oplus 2^t} \end{pmatrix}. \quad (6)$$

The 2^n indices form 2^{n-1} disjoint pairs, so the total work is $O(2^n)$ per gate rather than $O(4^n)$ [11], [12]. Controlled gates extend this naturally: the pair update of (6) is applied only when the control bits of the index are 1, leaving all other amplitudes untouched. This pairwise update is exactly the multi-qubit generalization of the single-qubit product in (4), with the same 2×2 matrix G acting on a different pair of amplitudes per index.

Measurement of a basis state $|i\rangle$ yields outcome i with probability

$$P(i) = |\alpha_i|^2, \quad (7)$$

after which the state collapses to the sampled outcome [23], [24]. Repeated-shot sampling repeats this draw, and the empirical frequencies form the measurement histogram shown in the UI.

Implementation-wise, amplitudes are stored in JavaScript typed arrays (`Float64Array`), with real and imaginary parts interleaved or paired [39]. Complex arithmetic is implemented once and reused throughout the core; the

dependency order of Section VI ensures that no module above the complex-number layer duplicates this arithmetic. The asymptotic argument above is the only performance claim made in this proposal; no benchmark numbers are stated.

VIII. VALIDATION PLAN

Validation proceeds in six hierarchical layers:

- 1) **Complex Arithmetic:** Unit tests verify scalar primitives (e.g. addition, multiplication, complex conjugation, modulus, etc.) against exact analytical values [38].
- 2) **Canonical State Transformations:** Single-qubit operations ($H|0\rangle$, $X|0\rangle$, $Z|1\rangle$) are tested to ensure they produce textbook state amplitudes [6].
- 3) **State Invariants and Normalization:** State validity checks ensure that all gate applications preserve unitarity and norm conservation, enforcing $|\alpha|^2 + |\beta|^2 = 1$ [6].
- 4) **Multi-Qubit Circuits and Algebraic Identities:** Tests verify entanglement generation via Bell and GHZ circuits [20], [21], validate algebraic relations such as $H^2 = X^2 = I$, and reserve quantum teleportation [22] as a composite integration benchmark once basic circuits stabilize.
- 5) **Algorithmic Benchmarks:** High-level implementations (Deutsch–Jozsa, Grover’s search, Shor’s Algorithm and the Quantum Fourier Transform) are evaluated against known theoretical results, such as the theoretical success rate of Grover’s search at peak amplification and exact QFT basis transformations [14], [15], [17].
- 6) **Differential Testing via Cross-Library Verification:** Comprehensive validation is conducted against established reference frameworks [3]–[5]. We define $m = \sum_{op} k_{op}$, where k_{op} is the qubit arity required by each implemented gate, relation, and state function. For each arity factor, test states are generated across six amplitude composition profiles for (α, β) : fully complex, purely real, purely imaginary, and the three asymmetric pairs (complex–real, complex–imaginary, and imaginary–real, with randomly swapped component assignments). These $6m$ states are serialized to JSON, executed across all supported operations, and the resulting state vectors are benchmarked against identical runs executed in reference libraries [3], [4].

The simulation core is designed to run with no UI attached, so each layer is exercisable by a script-driven test runner without any manual interaction. This separation is what makes a meaningful validation plan feasible within the project’s scope.

IX. DEVELOPMENT PLAN

Development is organized into eight phases.

- 1) *Phase 1* fixes the architecture and the data-model specification.
- 2) *Phase 2* implements the mathematical core: complex numbers and the quantum state.
- 3) *Phase 3* adds the circuit engine and measurement.

- 4) *Phase 4* builds the minimal drag-and-drop UI and begins development of explainer AI described in Section V.
- 5) *Phase 5* adds visualization (probabilities, amplitudes, state-vector text, histogram).
- 6) *Phase 6* implements the algorithms.
- 7) *Phase 7* adds persistence in JSON and URL form.
- 8) *Phase 8* is reserved for polish.

X. CONCLUSION

This proposal describes a browser-based quantum circuit simulator that runs without quantum hardware and without existing SDKs. The simulator represents states explicitly, applies gates to their target qubits at $O(2^n)$ per-gate cost, and aims to ship at least three canonical algorithms, Deutsch–Jozsa, Grover, Shor and the Quantum Fourier Transform, verified against their theoretical results [14], [15], [17]. A custom drag-and-drop editor drives a UI-free simulation core, and the team aims to add stepwise inspection, reusable subcircuits, entanglement highlighting, and an AI module with a swappable backend. Overall, our vision is to create the perfect open-access, easy-to-use virtual laboratory that both teaches the next generation of students and researchers and supports the current.

REFERENCES

- [1] C. Gidney, “Quirk: A drag-and-drop quantum circuit simulator that runs in your browser,” GitHub repository, Apache-2.0 license, 2016. [Online]. Available: <https://github.com/stlanc/quirk>
- [2] OpenAI, “ChatGPT (GPT-4o architecture),” Large language model, 2024. [Online]. Available: <https://chatgpt.com>
- [3] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” 2023, doi: 10.5281/zenodo.2573505.
- [4] Cirq Developers, “Cirq: A python framework for creating, editing, and invoking noisy intermediate scale quantum (NISQ) circuits,” 2022, doi: 10.5281/zenodo.6599601.
- [5] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, M. S. Alam, S. Ahmed, J. M. Arrazola, C. Blank, A. Delgado, Z. Jahangiri *et al.*, “PennyLane: Automatic differentiation and machine learning of quantum computers,” *arXiv preprint arXiv:1811.04968*, 2018.
- [6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge, UK: Cambridge University Press, 2010.
- [7] P. Kaye, R. Laflamme, and M. Mosca, *An Introduction to Quantum Computing*. Oxford, UK: Oxford University Press, 2007.
- [8] N. D. Mermin, *Quantum Computer Science: An Introduction*. Cambridge, UK: Cambridge University Press, 2007.
- [9] IBM Quantum, “IBM Quantum Composer,” IBM Corporation, 2021. [Online]. Available: <https://quantum.ibm.com/composer>
- [10] P. Migdal, K. Jankiewicz, P. Gradiński, C. Decaroli, and P. F. Cohadon, “Visualizing quantum mechanics in an interactive simulation virtual lab,” *American Journal of Physics*, vol. 90, no. 11, pp. 864–876, 2022.
- [11] G. F. Viamontes, I. L. Markov, and J. P. Hayes, *Quantum Circuit Simulation*. Dordrecht, Netherlands: Springer Science & Business Media, 2009.
- [12] K. De Raedt, K. Michielsen, H. De Raedt, B. Trieu, G. Arnold, M. Richter, T. Lippert, H. Watanabe, and N. Ito, “Massively parallel quantum computer simulator,” *Computer Physics Communications*, vol. 176, no. 2, pp. 121–136, 2007.
- [13] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, “qHiPSTER: The quantum high performance software testing suite,” *arXiv preprint arXiv:1601.07195*, 2016.
- [14] D. Deutsch and R. Jozsa, “Rapid solution of problems by quantum computation,” *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, vol. 439, no. 1907, pp. 553–558, 1992.

- [15] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [16] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134.
- [17] D. Coppersmith, "An approximate Fourier transform useful in quantum factoring," *arXiv preprint quant-ph/0201067*, 2002.
- [18] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Physical Review A*, vol. 52, no. 5, p. 3457, 1995.
- [19] T. Toffoli, "Reversible computing," in *Automata, Languages and Programming*, J. W. de Bakker and J. van Leeuwen, Eds. Berlin, Heidelberg: Springer, 1980, pp. 632–644.
- [20] J. S. Bell, "On the Einstein Podolsky Rosen paradox," *Physica Physique Fizika*, vol. 1, no. 3, p. 195, 1964.
- [21] D. M. Greenberger, M. A. Horne, and A. Zeilinger, "Going beyond Bell's theorem," in *Bell's Theorem, Quantum Theory and Conceptions of the Universe*. Springer, 1989, pp. 69–72.
- [22] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, p. 1895, 1993.
- [23] M. Born, "Zur Quantenmechanik der Stoßvorgänge," *Zeitschrift für Physik*, vol. 37, no. 12, pp. 863–867, 1926.
- [24] J. von Neumann, *Mathematical Foundations of Quantum Mechanics*. Princeton, NJ: Princeton University Press, 1955.
- [25] F. Bloch, "Nuclear induction," *Physical Review*, vol. 70, no. 7-8, p. 460, 1946.
- [26] U. Fano, "Description of states in quantum mechanics by density matrix and operator techniques," *Reviews of Modern Physics*, vol. 29, no. 1, p. 74, 1957.
- [27] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, "Quantum entanglement," *Reviews of Modern Physics*, vol. 81, no. 2, p. 865, 2009.
- [28] W. K. Wootters, "Entanglement of formation of an arbitrary state of two qubits," *Physical Review Letters*, vol. 80, no. 10, p. 2245, 1998.
- [29] Y. Su, H. C. Wu, and C. K. Cheng, "Interactive quantum circuit debugging via state inspection and runtime assertion," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2021, pp. 312–320.
- [30] A. Miranskyy, L. Zhang, and J. Denzinger, "Testing and debugging quantum programs," in *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 2021, pp. 46–50.
- [31] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," *arXiv preprint arXiv:1707.03429*, 2017.
- [32] L. Brito, E. Sanz, and R. M. Cruz, "Large language models for quantum circuit synthesis and explanation," *IEEE Transactions on Quantum Engineering*, vol. 5, pp. 1–12, 2024.
- [33] Z. Li, X. Wang, and Y. Gao, "AgentQ: Synthesizing and explaining quantum circuits with language model agents," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2024, pp. 105–114.
- [34] G. Li, L. Zhou, and M. Ying, "Towards pedagogical assistance in quantum circuit design using generative models," in *Proceedings of the ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2023, pp. 580–586.
- [35] Ollama, "Ollama: Get up and running with large language models locally," 2023. [Online]. Available: <https://github.com/ollama/ollama>
- [36] P. A. M. Dirac, "A new notation for quantum mechanics," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 35, no. 3, pp. 416–418, 1939.
- [37] A. Graham, *Kronecker Products and Matrix Calculus with Applications*. Chichester, UK: Ellis Horwood Ltd., 1981.
- [38] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge, UK: Cambridge University Press, 2007.
- [39] ECMA International, *ECMAScript 2023 Language Specification*, Standard ECMA-262, 14th ed., Jun. 2023. [Online]. Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>